Neural Comput & Applic (2011) 20:59–78
DOI 10.1007/s00521-010-0442-0

ORIGINAL ARTICLE

A neural network proxy cache replacement strategy and its implementation in the Squid proxy server

Sam Romano · Hala ElAarag

Received: 10 June 2009 / Accepted: 3 September 2010 / Published online: 24 September 2010
© Springer-Verlag London Limited 2010

Abstract As the Internet has become a more central aspect for information technology, so have concerns with supplying enough bandwidth and serving web requests to end users in an appropriate time frame. Web caching was introduced in the 1990s to help decrease network traffic, lessen user perceived lag, and reduce loads on origin servers by storing copies of web objects on servers closer to end users as opposed to forwarding all requests to the origin servers. Since web caches have limited space, web caches must effectively decide which objects are worth caching or replacing for other objects. This problem is known as *cache replacement*. We used neural networks to solve this problem and proposed the Neural Network Proxy Cache Replacement (NNPCR) method. The goal of this research is to implement NNPCR in a real environment like Squid proxy server. In order to do so, we propose an improved strategy of NNPCR referred to as NNPCR-2. We show how the improved model can be trained with up to twelve times more data and gain a 5–10% increase in Correct Classification Ratio (CCR) than NNPCR. We implemented NNPCR-2 in Squid proxy server and compared it with four other cache replacement strategies. In this paper, we use 84 times more data than NNPCR was tested against and present exhaustive test results for NNPCR-2 with different trace files and neural network structures. Our results demonstrate that NNPCR-2 made important, balanced decisions in relation to the hit rate and byte hit rate; the two performance metrics most commonly used to measure the performance of web proxy caches.

Keywords Web caching · Proxy server · Squid · Neural network · NNPCR · Cache replacement strategies

S. Romano · H. ElAarag (✉)
Department of Mathematics and Computer Science, Stetson University, 421 N Woodland Blvd, Deland, FL 32723, USA
e-mail: helaarag@stetson.edu

S. Romano
e-mail: sromano@stetson.edu

1 Introduction

The web has become the most important source of information and communication for the world. Due to the large increase in Internet traffic, some servers have been designed to be proxies and utilize the process of caching, storing data temporarily to help decrease the amount of traffic at any given time. Proxy servers are designed with three goals: decrease network traffic, reduce user (client) perceived lag, and reduce loads on the origin servers [1, 2].

Several problems emerge from the fact that there is a limited amount of hard disk space that proxies can save objects onto. Some examples of decision-making problems are *cache placement*, *cache consistency*, and *cache replacement*. Our main focus is the *cache replacement* problem, the process of evicting objects in the cache to make room for new objects.

There are many replacement strategies to consider when designing a proxy server. The most commonly known cache replacement strategies are Least Frequently Used (LFU) and Least Recently Used (LRU). Until 2003, there had been no survey of known web cache replacement strategies. However, Podlipnig et al. [1] has done well to not only list well-known strategies, but also categorize the strategies into five groups: *Frequency Based*, *Recency Based*, *Frequency/Recency Based*, *Function Based,* and *Randomized*.

As we will demonstrate later in this paper, many strategies suffer from not being able to adapt dynamically to ever changing levels of request stream characteristics such as temporal and spatial locality. *Temporal locality* is the measure of how likely an object is to appear again in a request stream after being requested within a time span, while *spatial locality* is the likelihood that an object will appear again based on how often it has been seen before [2]. In a previous study [3], we demonstrated that many strategies fail to separate the characteristics of the request stream from the characteristics of individual web requests and as such also miss one, two, or all three goals of a web proxy server in comparison with other strategies.

Many researchers are interested in web caching and its associated problems [4–7]. Oritz et al. [8] exploited the large infrastructure of today's web proxy caching systems to interactively transmit JPEG2000 images, while Liu et al. [9] used them for media streaming. Luo et al. [10] focused on making proxy caching work for database-backed Web sites. Many proposals can be found in the literature regarding web cache decision processes. Houtzager et al. [11] proposed an evolutionary approach to find an optimal solution to the web proxy cache placement problem, while Aguilar and Leis [12], for example, addressed the replacement problem. Fagni et al. [13] propose a static dynamic cache. They store the most popular web queries in a static, read-only portion of a cache. The remaining cache entries are dynamic and store other queries that cannot be satisfied using the static cache. The dynamic cache is managed by *any replacement policy*. In 2009, Kaya et al. [14] devised an admission-control policy to screen documents based on a mathematical expression that is function of average delay per request. They use this policy to identify cacheable and non-cacheable documents and then use LRU for cache replacement.

With the widespread of mobile devices, wireless environments are specialized environments where cache replacement policies are used to improve performance. To mention a few, Katsaros and Manolopoulas [15] propose a policy to not only reduce latency but conserve scarce network resources in mobile wireless networks. Kumar et al. [16] implement a prototype that utilizes multilayered caching techniques to improve performance of mobile devices.

Many cache replacement policies use artificial intelligence techniques for decision making. Sabeghi et al. [17] and Calzarossa [18] use fuzzy logic for improving cache replacement decisions. A survey on applications of neural networks and evolutionary techniques in web caching can be found in [19]. Khalid proposed a neural network-based cache replacement scheme [20] called KORA. KORA is based upon earlier work by Pomerene et al. [21], which uses shadow lines. Shadow lines refer to cache lines that are not currently active but have been active in the past. KORA and KORA-2 [22] use neural networks to designate cache lines as shadow lines which then receive preferential treatment. Outside of this distinction, the KORA algorithms use LRU. Obaidat and Khalid [23, 24] proposed other algorithms for cache replacement using neural networks that are based on KORA. However, KORA addresses traditional cache replacement. Algorithms used in traditional cache replacement form the basis for replacement algorithms used in web caching, but have received extensive modification. Web proxy cache workloads differ from traditional caching due to variable object size and bursty request sequences, among other things [1, 25–30].

Neural networks were applied to the web proxy cache replacement problem in 2006 as a novel cache replacement strategy by Jake Cobb and Hala ElAarag [31]. The resulting framework was named the Neural Network Proxy Cache Replacement (NNPCR) method. Simulations suggested that this approach is a promising approach to web proxy cache replacement. However, it was unknown how well NNPCR would fair against other strategies in a practical environment. In this research, we propose several improvements to enhance the performance of NNPCR [32]. We implemented NNPCR in the Squid Proxy Cache version 3.1 as well as the function-based strategy, M-Metric [33] to see how they would fair against the existing strategies implemented in Squid.

The rest of this paper is structured as follows. Section 2 covers definitions related to web caching and the Squid Proxy Cache. We describe current cache replacement strategies implemented in Squid, along with M-Metric and NNPCR as proposed by Cobb and ElAarag in Sect. 3. Section 4 discusses our improved implementation of NNPCR, known as NNPCR-2, its training, and lastly its implementation in Squid version 3.1. Our training results of NNPCR-2 are presented in Sect. 5, as well as the criteria for measuring the training performance. Section 6 describes the testing setup for Squid and how each strategy was benchmarked. Section 7 presents the results and observations of our tests of various cache replacement strategies covered in this paper including NNPCR-2. We conclude our results in Sect. 8, and lastly follow up with future work to be done in Sect. 9.

2 Background information

It is necessary to have a clear and precise definition on when a web object is allowed to be cached. A thorough, well-defined definition allows web cache users to understand what requests they make could potentially be cached, and as well, is necessary for system administrators as a

good tool in improving Quality of Service (QoS) for end users. For this paper, a *web object* is a term used for all possible objects (HTML pages, images, videos, etc.) transferred through the HTTP protocol that can be stored in a proxy cache [1].

2.1 Web request

A web request is a reference made through the HTTP protocol to a web object, primarily referenced by their Uniform Resource Identifier (URI).[1] Requests are also identified by the size of the requested web object from the origin server (at the time the request was made), an HTTP return code, and the time the proxy received the request. We define a *cacheable request* to have the following criteria:

- There must be a defined size, in bytes, for the request, and that size must be less than some threshold typically set by a network administrator and greater than zero.
- The request must be a GET or HEAD request and its status code must be one of the following, as set by the HTTP 1.1 Protocol [34]: 200, 203, 206, 300, 301, or 410.

Separate from *cacheable request*, we also ignore any requests with URIs containing "/cgi-bin/" as well as any URIs that are queries (those that contain a question mark in their URI after the last "/").

Once a request is known to be cacheable and is received by the proxy, several things will occur in a sequential order. In a basic proxy server model, if a *cache hit* occurs, then the object being referenced is in the cache and the data can be copied and sent back to the client. On a *cache miss*, when no object in the cache matches the request, the web object will be retrieved from the origin server and the *cache placement strategy* decides whether the object will be placed into the cache. If there is not enough room for the new object to be added, then the *cache replacement strategy* is invoked. However, in order to understand how these strategies work, we will define several object characteristics these strategies consider.

2.2 Characteristics of web objects

Web objects are identified by several different characteristics. Each replacement strategy requires usually a small subset of characteristics; however, all web objects must be identified by their URI since this is the only unique characteristic. The most important characteristics are [1]:

- Recency: information related to the time the object was last requested.
- Frequency counter[2]: number of requests to the object.
- Size: the size, in bytes, of the web object.
- Cost: the 'cost' incurred for fetching the object from the origin server. Also known as the miss penalty.
- Request value[3]: the benefit gained from storing the object in the cache. This is generally a heuristic based on other characteristics, since an actual request value of an object cannot be determined without a priori information.
- Expiration time: Also generally a heuristic, either defined by the proxy or by the origin server of when the object will become stale and should be removed or refreshed in the cache. Also known as the time-to-live (TTL).

Most strategies use a combination of these characteristics to make their replacement decisions. The expiration time is the only characteristic mentioned that was not utilized in our cache replacement mechanism and is primarily referenced when dealing with the problem of *cache consistency*, which is out of the scope of this research. The request value is an abstract characteristic, primarily used by *Function-based* strategies, and defined by a characteristic function that pursues a total, well-defined ordering of the objects. Note that this characteristic will be set by M-Metric and NNPCR as shown in Sect. 3.2.

2.3 Squid and current caching problems

Squid [35] is an open source proxy cache project now maintained by volunteers, originally supported through an NSF grant, and now fueled purely by donations. Squid currently supports HTTP 1.0 protocol and is almost HTTP 1.1 compliant. One difference between HTTP 1.0 and 1.1 protocols is the cacheable request definition. For instance, partial requests (return code 206) are not allowed to be cached in Squid, as opposed to HTTP 1.1 compliant software, which is allowed to be cached. Through Squid's configuration files and easy compilation steps, one can control almost any aspect of the cache including the replacement strategy, and how the proxy logs requests, such as keeping a log of the requests that Squid automatically saves to the hard disk. In the early 1990s, the Squid project came about from a fork of the Harvest Cache

[1] For HTTP Requests, this is also known as the Uniform Resource Locator (URL), but URI is the better, generic term to use here.

[2] Many literatures also refer to this as the access counter; we found this to be vague, as it was commonly ambiguous with terms such as disk access, or access of the object's information during sorting methods.

[3] Essentially, any characteristic could be the request value, if the algorithm makes its decision based on one variable that has a total, well-defined ordering.

Daemon. The other fork existing today is Netapp's Netcache [35], which is a proprietary software package.

Originally, the NSF funded Squid in order to promote research in the 1990s before the conception of broadband, DSL, and other technologies that lead to a huge increase in bandwidth. As far as many businesses and researchers were concerned, the web caching problem was essentially deemed 'solved' in the late 1990s and early part of this decade [1]. Most of these strategies were simple due to the lack of CPU power, available RAM, and especially available storage. In fact, it was rare to see a proxy server that had more than 2 gigabytes of space devoted to the cache. Today, this amount of disk space is trivial.

However, the decrease in costs for bandwidth, increase in CPU power, greater storage capacities, and higher demand for download speed have lead to bandwidth problems again especially for web publishers who host major Web sites. Increase in the use of dynamic pages as well is leading to yet another strain in the bandwidth provided by today's Broadband/WAN technologies. These new dilemmas have lead many companies, businesses, and even Internet Service Providers (ISPs) to turn to research in caching. However, many strategies authored in the late 1990s and early part of this decade are failing in being able to dynamically adapt to request streams and the constantly changing environment that the Internet has now become.

Although Podlipnig et al. [1] has provided a comprehensive survey of past strategies; the literature also noted that the caching problem is far from solved with the increase in dynamic web content. Essentially, most strategies that are used in proxy cache software such as Squid are no longer "good-enough" strategies today.

Currently, Squid provides a choice between LFU Dynamic-Aging (LFU-DA) [25], Greedy-Dual-Size-Frequency (GDSF) [25], a heap-based variation of LRU, and by default enables a linked-list version of its LRU variant. The result from our tests shown later demonstrates that LRU, while a great base algorithm, in fact has a lower performance than LFU-DA and GDSF. GDSF also operates under a different *cost model* than we simulated in our previous research [3] and has unique effects on the algorithm. If the maximum size threshold for objects is increased, LFU-DA's performance can actually be changed as well.

Due to these recent developments, Squid is a good test bed for NNPCR. As the need for web proxy caching increases, Squid grows more popular in use and in support, and the added support and popularity has lead to a rather stable product. In fact, as early as 1999, Dilley et al. [36] from Hewlett Packard's Internet Systems and Applications Laboratory were responsible for adding LFU-DA and GDSF along with the heap-variation of LRU to Squid. Their research sheds light on how to go about implementing replacement policies for Squid and different configurations to test.

LFU-DA, GDSF and LRU are implemented in Squid by using a general class known as the heap-based replacement strategy. Here, the assumption is to induce a total ordering to the current objects in the cache. Due to heap implementation, objects could be ordered based on a request value with a complexity of $O(n\log_2(n))$. Squid's heap replacement strategy is designed around callback functions (a strategy signified by pointing certain components or methods to functions responsible for giving the proper output); all one has to do is implement a new 'key' generation callback function and allow Squid to be configured to point to that function. This will be covered later in more detail in the implementation section.

3 Related replacement strategies

In Sect. 1, we introduced the classifications set by Podlipnig et al. [1]. These strategies arise by taking notice to the primary characteristic which the strategy makes its decision upon. The first two groups, *Recency* and *Frequency*, are based mainly on Least-Recently Used (LRU) and Least-Frequently Used (LFU), respectively. *Frequency/Recency* strategies incorporate a mixture of an object's recency and frequency information together along with other characteristics to refine LRU and LFU. *Function-based* strategies have some defined method that accepts certain pre-defined parameters defining a request value to order the objects in the cache. The last group, *Random*, essentially picks an object in a non-deterministic manner [1].

Table 1 contains commonly used variables and their descriptions, which will be referred to throughout the rest of this paper. If a strategy uses a variable not defined in the table, then it will be defined in their summary. Any use of the logarithmic function, symbolized as log, is assumed to be of base 2, unless otherwise specified.

Table 1 Characteristic symbols

Variable	Description
S_i	Size of an object i
T_i	Time object i was last requested
ΔT_i	Time since object i was last request
F_i	Frequency counter of object i
ΔF_i	Number of references to occur since last time object i was referenced
l_i	Access latency for object i
C_i	Cost of object i
R_i	Request value of object i
M	Size of the cache

3.1 Squid's implemented cache replacement strategies

Squid comes with three strategies already implemented: *LRU*, *LFU Dynamic Aging* (*LFU-DA*), and *Greedy-Dual-Size-Frequency* (*GDSF*). As explained in Sect. 2.3, these strategies are implemented within a heap-based class, and so are sorted based on a request value assigned to each object and have an order of complexity of $\Theta(n \lg n)$. In the rest of this section, we will provide a summary of each of these three strategies.

- *LRU*: One of the most commonly used strategies in many areas of cache management. This algorithm removes the least recently used (referenced) object, or in other terms the object with the largest ΔT_i or smallest T_i. A simple linked list (queue) allows this algorithm to be efficiently implemented and is referred to as a *LRU list*.
- *LFU-DA* [25]: Standing for Least Frequently Used—Dynamic Aging, *LFU-DA* attempts to remove cache pollution induced by *LFU*. A common problem that occurs is with *hot* objects. These objects become popular incredibly quick, and so their frequency counts skyrocket, and then are no longer requested. In a complete *LFU* strategy, these objects would expire long before the replacement method would remove them. As such, *LFU-DA* ages the frequency counts of older objects in order to account for this issue. Upon a request to object i, its value, R_i, is calculated as

  $$R_i = F_i + L \tag{1}$$

  where L is a dynamic aging factor. Initially L is set to 0, but upon the removal of an object i, L is set to R_i. This strategy removes the object with the smallest R_i value.
- *GDSF* [25]. An extension of GD-Size, this uses the frequency information and cost information as well as size to define a request value. The request value is defined as

  $$R_i = \frac{F_i * C_i}{S_i} + L \tag{2}$$

  L is an aging factor used exactly like in LFU-DA. The advantage in this method is that objects which are beneficial to caching (typically with higher latencies) can be cached and decrease the network bandwidth. However, in Squid, it assumes what is called the *uniform cost model*, that is, for all objects, $C_i = 1$. As a result, the implementation of GDSF in Squid loses this advantage.

3.2 Implemented cache replacement strategies

In this section, we provide a description of the strategies we implemented within Squid version 3.1. We first present M-Metric [33], a function-based method, that was selected because of the interesting way it combines several characteristics together to make an assessment of the value of an object. The second strategy we implemented was NNPCR, as it was the focus of this research.

- *M-Metric* [33]: This strategy takes three parameters: f, s, and t. With these in mind, it defines the request value as

  $$R_i = F_i^f * S_i^s * \Delta T_i^t \tag{3}$$

  where f should be positive as to give weight to popular objects. A positive s value will give higher weight to larger objects, while a negative value will give higher weight to smaller objects. t reflects how much recency is taken into account. A positive value gives weight to older objects, while a negative value will result in recent objects taking precedence over older ones. Based on the parameter values, this algorithm will decide exactly like LRU ($f = 0,\ s = 0,\ t < 0$), LFU ($f > 0,\ s = 0,\ t = 0$), and SIZE ($f = 0,\ s < 0,\ t = 0$). One large difference between M-Metric and most other function-based methods, however, is the lack of an aging factor. As a result, in an ideal cache setup, the value of an object in M-Metric should be updated each time a decision on the heap must be made (right before removing an object from the cache).
- *NNPCR* [31]: NNPCR is a web proxy cache replacement scheme which incorporates a neural network. The network is a two-hidden layer feed-forward artificial neural network which falls under the Multilayer Perceptrons (MLP) model. NNPCR sets the connection weights through supervised learning with back-propagation. It trains the network using subsets of the trace data sets. Additionally, it uses separate subsets of the trace data sets for verification of network generalization. Error is calculated by using the known classification of each request for target output. Once the neural network has been created and properly trained, NNPCR is used to handle cache replacement decisions. All candidate replacement objects were rated by applying either frequency and recency information alone or those parameters plus size, across the inputs of the network. An object is selected for replacement based on the rating returned by the neural network. NNPCR aims to take advantage of neural networks' universal approximation and generalization properties. The neural networks created by NNPCR were able to classify both training and verification sets with Correct Classification Ratio (CCR) values in the range of 0.85–0.88, which indicates effective generalization. NNPCR derives workload assumptions from the training data (partial workloads) rather than holding a static set of assumptions or requiring the user to set parameters that dictate which

Table 2 NNPCR fixed parameters

Parameter	Value
Neural network model	Fully connected, feed-forward MLP model
Hidden layers	2
Normalized input range	[−0.9, 0.9]
Activation function	Sigmoid
Error function	Cross-entropy error
Inputs	2 (recency, frequency) or 3 (recency, frequency and size)
Outputs	1 (likelihood of a future request)
Weight update algorithm	Batch-mode back-propagation
Training metric	Correct classification ratio (CCR)

Table 3 NNPCR variable parameters

Parameter	Suggested value(s)
Learning rate	0.005–0.05
Momentum rate	0.2–0.6
Nodes per hidden layer	3–10
Training epochs allowed	500–2000

assumptions behavior will reflect. It has a range of different inputs which it can be trained for. Cobb et al. [31] suggested several ranges for parameters to use when training the neural networks. Fixed parameters are presented in Table 2, while ranges for variable parameters are presented in Table 3. During training, a target value of 1.0 is assigned to a pattern if the particular web object is requested within the training set afterward; otherwise, it is assigned a target value of 0.

4 NNPCR-2

While simulations demonstrated that NNPCR would achieve high performance rates, there were several limitations we encountered in this research. NNPCR had a 2-h training set and then a 2-h testing set. More favorably, training and testing should be for longer periods. In this way, we can ensure that NNPCR-2 would perform well for a long period of uptime.

More than anything else, the recency input is heavily related to the running time of the strategy. This even affects the target output for training the strategy. For instance, if an object has a mean time between requests of 4 h, is it worth it to cache the object? According to NNPCR, each of these requests when trained by the neural network would have a target value of 1.0 despite the fact that the object would most likely expire from the cache well before.

NNPCR was also able to read in the trace files prior to running each training/simulation session and found the maximum recency, frequency, and size values. This is clearly impossible in a running implementation. So clearly, we must define thresholds for each of the inputs and cap them if they happen to go above so that we can normalize our values into the input range [−0.9, 0.9] still. However, by adding more parameters/thresholds, we now make NNPCR more parameterized than before, taking away from its dynamic characteristics. In the following sections, we will describe the new method of normalizing our inputs, as well as redefining our target value.

4.1 NNPCR-2 implementation details

In this section, we provide the details for our implementation of NNPCR-2 and the revisions we applied to the original NNPCR.

4.1.1 Input normalization methods and the sliding window

There are three inputs that NNPCR requires recency, frequency, and size. Of the three, recency is the most troublesome of the parameters to normalize. Clearly, it is possible that the highest recency value could be up to a day, week, or even more. However, it is unlikely that most objects will see a recency value that high, which means when we normalize our value, most objects will clutter around −0.9, which will not allow the neural network to be able to generalize very well. It may get to the point where the neural network would just concentrate on the other two inputs, namely, frequency and size, respectively, and zero out the weights to the recency input node if it did not tell enough information. By zeroing out the weights the neural network would have nullified the purpose of using recency as an indicator for cacheability.

To counter this issue, we decided to implement a sliding window around the time a request is made. With this in mind, we could train our neural network around this sliding window and redefine the semantic of our output value to go according to this sliding window. In NNPCR [31], we could say that the sliding window had a length of 2 h, since all the testing and training were done within 2 h. That means any output by the neural network was a judgment for within the next couple of hours.

To formally define what we mean by a sliding window, the system administrator would have to train and configure the neural network based on some sliding window length generally set in milliseconds for our training program and seconds for Squid version 3.1. The sliding window of a request is thus the time before and after when the request was made. Within this sliding window, we can examine the frequency count during training, and the recency time. As a

general rule of thumb, the sliding window should be around the mean time that an object generally stays in a cache. From here on, the sliding window length will be referred to as *SWL*. Other symbols for object characteristics are shown in Table 1.

With the sliding window in mind, we update the recency value at each request to an object. When an object is requested, we set the recency value as shown in (4), which means that the recency value is set to the maximum of SWL and ΔT_i *if object i was requested before* i.e. ΔT_i is greater than 0. Otherwise, it will be set to SWL.

$$\text{recency}(x) = \max(SWL, \Delta T_i) \tag{4}$$

With this method, we then normalize the values into the range of [−0.9, 0.9] by a typical linear normalization formula such as shown in (5). Where $\max_{\text{norm}} = 0.9$, $\min_{\text{norm}} = -0.9$, $\max_{\text{data}}$ and $\min_{\text{data}}$ are the maximum and minimum values in the data, respectively, x_{actual} is the value before normalization, while x_{norm} is the normalized value.

$$x_{\text{norm}} = \frac{x_{\text{actual}} - \min_{\text{data}}}{\max_{\text{data}} - \min_{\text{data}}} * (\max_{\text{norm}} - \min_{\text{norm}}) + \min_{\text{norm}} \tag{5}$$

For the frequency input value, it is relatively simple to decide on a threshold value. We must choose a frequency high enough that most objects will not ever reach it, but low enough so that the input values are dispersed evenly across the range [−0.9, 0.9]. In a previous study [3], we discussed how methods such as the *Cubic Selection Scheme* (CSS) [37] and *Pyramidal Selection Scheme* (PSS) [38] have a maximum frequency and size threshold to order objects into set classes. This method of choosing a frequency in conjunction with a size threshold value lead to some decent assumptions about various "classes" of objects based on their frequency and size class. As such, CSS and PSS perform quite well. We found that a maximum frequency of 128 was a good-enough range [3]. Thus, in this research, we capped any frequency inputs to 128.

As stated before, CSS and PSS also incorporated *object classes* based on their size and their frequency as well. An object *i* belonged to a class *j* if and only if $j = \log(S_i + 1)$. For NNPCR-2, we decided to divide objects into their size classes as well, as opposed to capping the threshold. This was done in hopes of pushing the neural network to generalize based on objects within their size classes. We then had to add a threshold to the size classes instead of the maximum object size. Since the size class is a logarithmic scale, the neural network has the ability to realize precise relationships with objects at lower sizes; between 2 and 1,024 bytes, there are 10 unique classes, while from 1,024 to 2,048, there are only 2 unique classes. This allows the neural networks to generalize about objects that are within class sizes, instead of specific sizes, which tends to be the case statistically as well. For this research, we chose a maximum size class of 22, which holds objects in the range $[2^{22}, 2^{23})$ bytes or up to 8 MB. Objects above 8 MB are essentially capped at that value and examined as if they were from the size 22 class.

4.1.2 Target value and the sliding window

Now that we have set what the inputs will be for each pattern, we must now figure out what the neural network output will signify, while still keeping it as some classification problem dealing with the likelihood of a future request. However, in order for the network output to signify anything, we must know what the target output value will be during the training process.

Building on the aforementioned sliding window on a particular request, the main method is to use the sliding window length of information in the past to make judgments on what will occur in the sliding window length ahead of the request. Since the previous section dealt with the sliding window prior to the request, then clearly the target value must deal with information following the request.

Semantically, the output node of the neural network must then represent the likelihood that a request occurs within the sliding window length *after* a request or particular pattern. Thus, simply, the target value of the neural network is 1.0 when there is in fact a request to the same object within the SWL afterward, and 0.0 otherwise. This is represented in (6).

$$\text{target}_p = \begin{cases} 1.0 & \text{if } \Delta T_i \leq SWL, \\ 0.0 & \text{otherwise} \end{cases} \tag{6}$$

Since the actual output of the neural network can also represent a probability, then anything that is above 0.5 can be classified as *cacheable* and anything below, as *unlikely cacheable within the SWL*. This is how we measure how the neural network correctly classifies a request. If we take the ratio of the number of patterns correctly classified as cacheable or unlikely cacheable to the total number of patterns, then we have the Correct Classification Ratio (CCR).

4.1.3 Frequency values and the sliding window length

When training the neural network, we estimated how the frequency values might be represented in a functional cache; we did not want to continually keep incrementing the frequency counter because of how the neural network was being trained. Assume the case where based on the SWL a target value of a particular request is set to 0. In an

actual implementation of NNPCR-2, the corresponding object might be removed as a result, and thus, its frequency counter would be reset when it was removed from the cache. When it was seen again, its frequency counter would be reinitialized to 1.

Thus, we assume that if ΔT_i is less than *SWL*, then frequency of object x will be incremented by 1. Otherwise, the frequency count would be 'decayed' by the number of sliding windows it is away from or set to 1, whichever is greater as shown in (7). Note that if a previous request did not occur before, then frequency(x) is set to 1.

$$\text{frequency}\,(x) = \begin{cases} \text{frequency}(x) + 1 & \text{if } \Delta T_i \leq SWL, \\ \text{MAX}\left(\frac{\text{frequency}(x)}{\lceil \frac{\Delta T_i}{SWL} \rceil},\ 1\right) & \text{otherwise} \end{cases} \tag{7}$$

The reason for the frequency decaying factor is that if the previous request was outside *SWL*, then clearly the target value of that previous request would be 0.0, and thus, unlikely to remain in the cache (we assume that the expected value of the frequency count in this scenario is proportional to the number of sliding windows that separate the requests). Conversely, if the previous request was within *SWL*, then chances are, it would remain in the cache in the optimal cache setting, and thus the frequency information would be retained fully.

4.2 NNPCR-2 training setup

For training NNPCR-2, we used 3 different trace files provided to us by IRCache.net [39]. The trace files used for training spanned a day's worth of recorded web requests, which is a lot longer than the training time of NNPCR. We also cleaned the trace files prior to feeding them through the neural network, allowing the neural network to be trained only on requests that would actually be cached in a web server as mentioned in Sect. 2.1.

We implemented a training program for NNPCR-2 separate from Squid. The reason for this is that we did not need an actual functioning cache to train the neural network. Rather, we just needed to process patterns, feed them forward, and then back propagate the error as demonstrated in Sect. 2.4.

The three locations of the trace files were Palo Alto, CA, New York, NY, and Urbana-Champaign, IL. They varied greatly in the number of requests and other request stream characteristics. The days that were used for each were arbitrarily chosen from the week of June, 10 to 17, 2007 and used as the training sets. The testing sets, which we will discuss in more detail later, were much longer (a week long of recorded tests) and more recent. The idea here was to test how well a long-term configuration of a neural network might pervade and observe how well it may generalize information about the request streams. By having several different trainings of other trace files, we were also able to test whether particular configurations were affected by the training sets used.

It is important to note here that although the trace files were from 2007, the results reported in this paper are still applicable to web 2.0 applications. Despite that some web 2.0 applications has dynamic objects, most of the web traffic has static content with file types as cascading style sheets, javascript files, images, etc. Most caching that occurs with dynamic content occurs on the browser side and does not occur from the standpoint of proxy servers. Our research studies cache replacement strategies designed for proxy cache servers concerned with static web content, as opposed to dynamic web content cached on the browser side. Thus, the experimental session in this paper is general and the proposed approach will always be valid as long as there are some static contents to web applications.

For NNPCR-2 the training metric, same as NNPCR, is known as the Correct Classification Ratio (CCR). CCR measures the number of patterns that the neural network correctly classifies (in this case, "cacheable" versus "unlikely cacheable"), to the total number of patterns seen in any given epoch.

We also followed the pocket algorithm [40], which saves the best weight set found during the testing procedure. This "best weight set" is measured by CCR. By saving this set throughout the training process, we can be assured that as time goes on, this best set will converge closer and closer to the global optimum. If we just used the last weight set at the end of the training, we may be using a weight set that was traversing an error plane and thus is not a good representation of what the neural network had learned in the past.

Another component of NNPCR-2's training, which differs greatly from NNPCR, is that we trained in online mode. When training in batch mode, we found that the values of CCR had huge discrepancies between consecutive epochs, which normally indicate that the learning rate is too high. Even turning down the learning and momentum rates, we were still unable to have the neural network stabilize to an acceptable CCR within 1,500 to 2,000 epochs. However, as we will show in Sect. 5, using online mode we are able to easily have the network converge to an acceptable CCR within 500 epochs.

Lastly, one of the most important additions to NNPCR-2 is the inclusion of the bias node. Reed et al. [40] discusses how the absence of a bias node means the input pattern (0, 0…0) will always intercept at the origin, or rather all outputs will be 0. Even with multiple hidden layers, this problem is always inherited. However, if we include a bias node, a node which is set to some constant, typically 1.0, then we can overcome the problem. As a result, on top of

the original 2 or 3 inputs, we also added another input node, which is always set to 1.0 in our neural network both in training and implementation.

4.3 Implementation of NNPCR-2 in Squid 3.1

The Squid implementation of NNPCR-2 has condensed the components of the neural network as much as possible—the only functionality that NNPCR-2 contains within Squid is that it can feed forward a pattern and produce an output, but includes none of the training functions such as back-propagation, target outputs. We implemented the entire algorithm within its own class called `NNPCR`. The entire implementation was done in C/C++, as that is the code base of Squid as well. Clearly, hard coding the sliding window length, size and frequency thresholds, inputs, weights, etc. were not an option in a well-designed implementation; thus, we implemented a flat file loader to read in a particular *nnpcr_squid* formatted file. This file could be written simply in any text editor, or by utilizing our training program, which produces the files as output.

We made a major change to how the heap generated keys for objects it held. In Sect. 2.3, we alluded to how Squid uses callback functions to call what is known as the `heap_key_gen_function`. Since the output of the neural network can be interpreted as a request value of the object, then the only addition was to implement a new key generating function. However, the current implementation did not support being able to pass parameters other than the `heap_age` or object which the key was being generated for. As a result, we added a new parameter to this function, `void *key_param`, which allowed us to pass an instance of the `NNPCR` class (initialized at the Squid startup process with the flat file loader) transparently as a parameter to the key generation function and without affecting the other replacement methods (since the key_param would just be null in those cases).

Interestingly, the `heap_age` parameter is saved and passed to all instances of this callback function. Referring to Sect. 3, the `heap_age` is actually the aging factor required for both LFU-DA and GDSF. Thus, upon an object removal, despite whatever key generation function was used, `heap_age` is set to the removed object's key value.

One reason for this aging factor is to quickly expire objects that have not been requested in a while, solving the *cache pollution* problem that can be brought on by the cache replacement strategy. When we first tested NNPCR-2 in Squid, our first emulations demonstrated that there was cache pollution occurring. In NNPCR, the 2-h span did not have to worry about cache pollution, because if an object was requested within that time span, it was most likely going to be seen again during the testing or not have long enough time to become a pollutant. As we will cover later, our tests covered an entire week's worth of web requests—certainly enough time for many objects to become a pollutant. Since NNPCR and NNPCR-2 output a value in the range [0.0, 1.0], objects with high values in the beginning will stay in the cache until they either expire or outlast their benefit effectively polluting the cache. Thus, we had to implement an aging factor to the output of NNPCR's neural network. The `heap_key_gen_function` is only called when objects are added into the cache or upon a cache hit.

5 NNPCR training results

As mentioned earlier, we trained the neural networks based on three trace files, each covering a day of recorded requests, and each from three different locations. We tested the effects of altering the number of hidden nodes, input nodes, learning rate (LR), and the sliding window length (SWL). We also tested altering the momentum rate, but it had such a small effect overall that it was not worth reporting. Thus, for all our tests, the momentum rate (MR) was set to 0.2.

The results will be broken up based on the effects of altering certain factors for each of the trace files. The trace file from Palo Alto, California will be abbreviated as PA, New York, New York as NY, and Urbana-Champaign, Illinois as UC. Once cleansed, the PA training set had about 9,457 patterns, NY had about 157, 838 patterns, and lastly the UC had 248,542 patterns—all of them spanned an entire day's worth of recorded web requests.

We refer to neural networks by their standard convention, which is a neural network with 2 inputs, 3 hidden nodes in each of its two hidden layers, and then 1 output node would be labeled as a 2/3/3/1 network. Since we have added a bias node, however, keep in mind that this does affect the input count.

Lastly, we set all the tests to 1,000 epochs. This may seem like overkill, but some of the best patterns did end up showing up later toward 1,000 as well. One observation we did notice in all our tests is that the neural network always converges quickly and tends to stabilize within 100 epochs in online mode, which means the error rate will continually keep decreasing at a steady rate. This was a huge difference from NNPCR. We were also able to achieve higher CCR values as well, consistently ending anywhere between 85.0 and ~92.0%. Our training results have previously appeared in [41]

5.1 Effect of hidden nodes and input nodes

For all tests, we tested the effect of changing the number of hidden nodes from 3 to 5 and finally to 10 in each hidden

layer. We received some surprising results, as it was dependent upon the training patterns used as opposed to finding a general pattern.

As shown in Fig. 1, increasing the number of hidden nodes for PA trace file generally had an increase in CCR. However, the increase was within hundredths of percents. The PA 3/10/10/1 case did gain quite a large increase in CCR especially around the 700th epoch. All curves, however, seem to take a logarithmic curve to them.

Figure 2 demonstrates that unlike the other trace files, a low hidden node count for NY trace file was unable to stabilize. At this point, we generally say that the neural network was over-generalizing and so the results may be able to stabilize if we turn down either momentum rate or learning rate. We can notice, however, that its counterparts, 3/5/5/1 and 3/10/10/1 had no trouble at all stabilizing around 90%. Clearly, here, the conclusion would be to just increase the number of hidden nodes as opposed to tweaking the learning and momentum rates.

For the UC trace in Fig. 3, we noticed that around 250 epochs, there was a rather quick dip by about a tenth of a percent, similar to what occurred in the NY trace file in the 3/3/3/1 network around 100 epochs; however, in both of the UC cases (3/3/3/1 and 3/5/5/1), the neural network was able to stabilize again. Overall, altering the number of hidden nodes for the UC-Test file had little to almost no effect on the CCR.

By adding in the size input to the neural network structure, altering the hidden inputs had expected results for both UC and NY—that is we can safely conclude generally increasing the number of hidden nodes increases the performance of NNPCR-2 during training. For the PA trace, it seems that the 4/5/5/1 network outperformed both the 4/3/3/1 and 4/10/10/1. In this case, by adding more hidden nodes, we have over generalized NNPCR-2. Although, performance wise, it is off by a tenth of a percent, which in the long run is about ten or so patterns, a completely minuscule value.

Examining the differences in Figs. 4, 5, 6 and 7, there is actually an increase in CCR when we use *less* input nodes (ignoring the size input that is). This relationship is opposite when examining the increase in the number of inputs nodes for both the NY and UC trace file when we keep all other parameters constant. However, increasing the input nodes for a particular input only gains about a tenth of a percent for the UC and NY files, which is a couple hundred requests, respectively. As a rule of thumb, using more inputs gains performance if there are a large quantity of patterns in the training set; more patterns allow the neural network to generalize on the third input—not enough patterns tends to lead the neural network in the wrong direction.

5.2 Effect of the sliding window length

The sliding window had some interesting results as well. While we performed 72 different instances of training the neural network with most of them testing the SWL, most of the results demonstrate the same relationship despite the parameters set. That is, as long as we keep the parameters constant, SWL has the same effect each time with those parameters. The following figures show only some of the tests we performed.

In Figs. 7, 8, 9, 10, 11, we see an interesting and rather large effect of the SWL on the 3/10/10/1 structure when trained on the PA trace. By decreasing the SWL, we actually lose performance by an entire percent (this correlates to a loss in classifying ~90 patterns). It may have been an oddity, but we observed this exact same performance decrease when all other factors (LR, MR, and

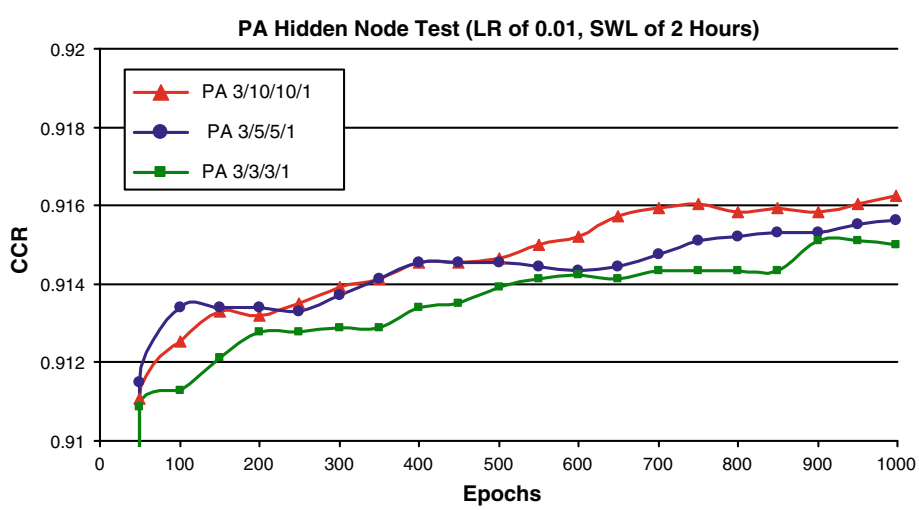

Fig. 1 Effect of hidden nodes for PA

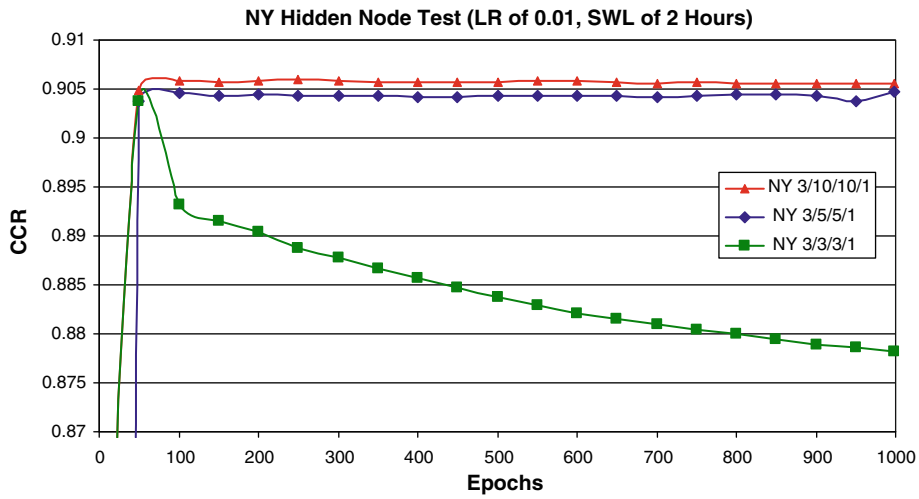

Fig. 2 Effect of hidden nodes for NY

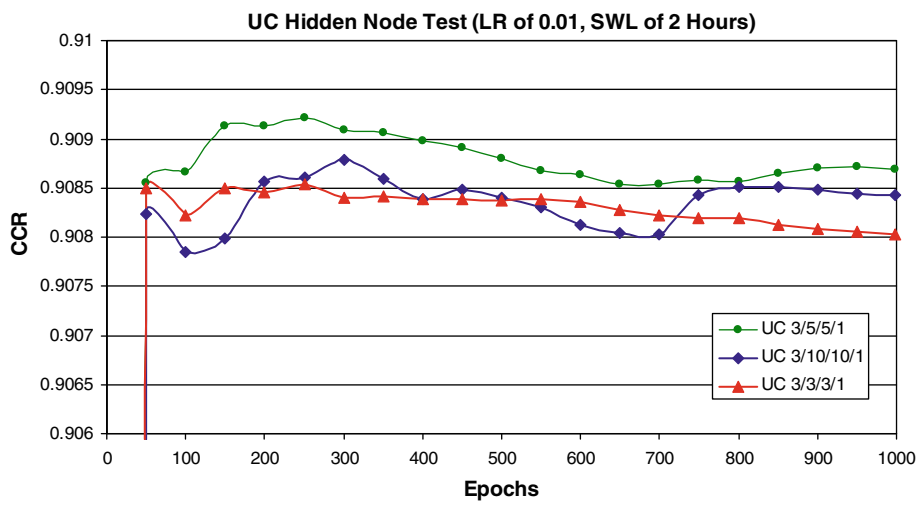

Fig. 3 Effect of hidden nodes for UC

structure) stayed the same. Thus, for sparse request streams such as the PA, we observe the general relation that a smaller sliding window decreases training performance.

In terms of the UC and NY training files, however, we gained almost a 2% performance increase in training when we dropped the SWL to a quarter of NNPCR trained with. Thus, on larger training sets, where the mean time between objects is much higher, decreasing sliding window length can actually increase performance. This may also allude to the fact that NNPCR-2 may be able to better handle denser request streams than sparse ones such as the PA-Test case.

5.3 Effect of the learning rate

To examine the effect of the learning rate on the neural network, we fixed all other parameters to what we found as their optimal value. This means for the PA NNPCR trainer, we used 3 input nodes (recency, frequency and bias), 3 hidden nodes in each hidden layer, and SWL of 2 h. For both UC and NY, we used the 4/10/10/1 structure with SWL of 30 min. Figures 13, 14, 15, 16 display the results of tweaking the learning rate at various levels.

In large regard, the results were exactly as we expected with tweaking the learning rate. A higher learning rate would lead to a less stable or more random traversal of the error plane (seen clearest in Fig. 12 with the UC 4/10/10/1 LR: 0.05 case), yet would lead to achieving the performance metric faster. Decreasing the learning rate smoothed out this curve, yet, unexpectedly did not decrease the CCR for any of the cases by much. In fact, we did not see the changes in performance as we had witnessed previously with the SWL. Overall, then, it seems that while learning rate can affect how the neural net learns over time, it is generally better to go for a smoother traversal of the error plane. Thus, for our experiment/tests in the next section, we used the case where LR = 0.001.

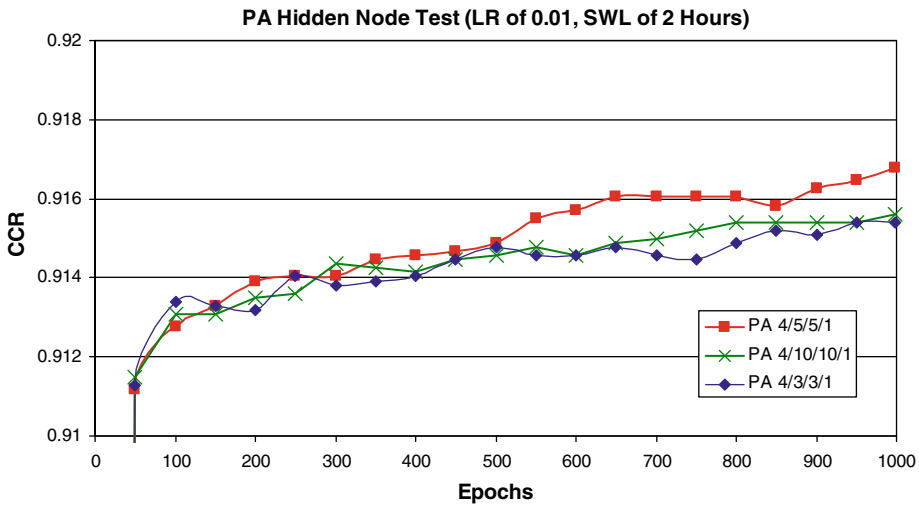

Fig. 4 Effect of hidden nodes for PA and 4 inputs

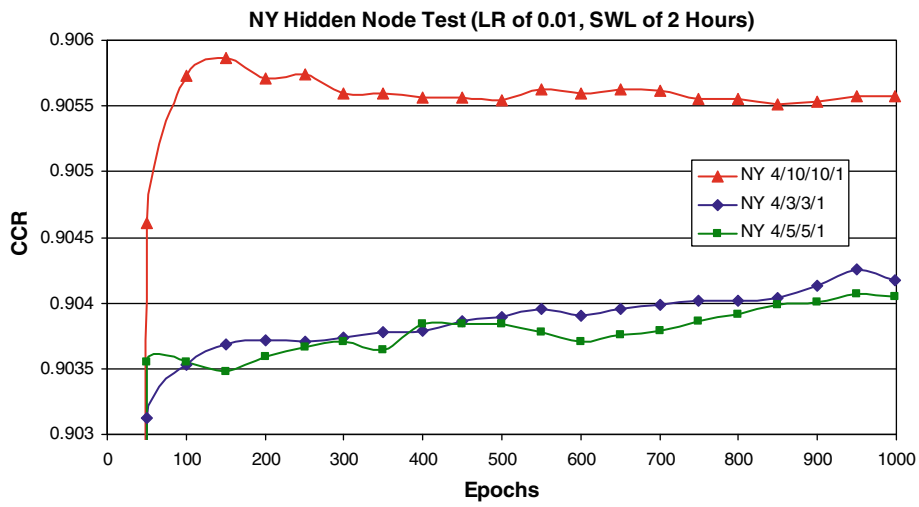

Fig. 5 Effect of hidden nodes for NY and 4 inputs

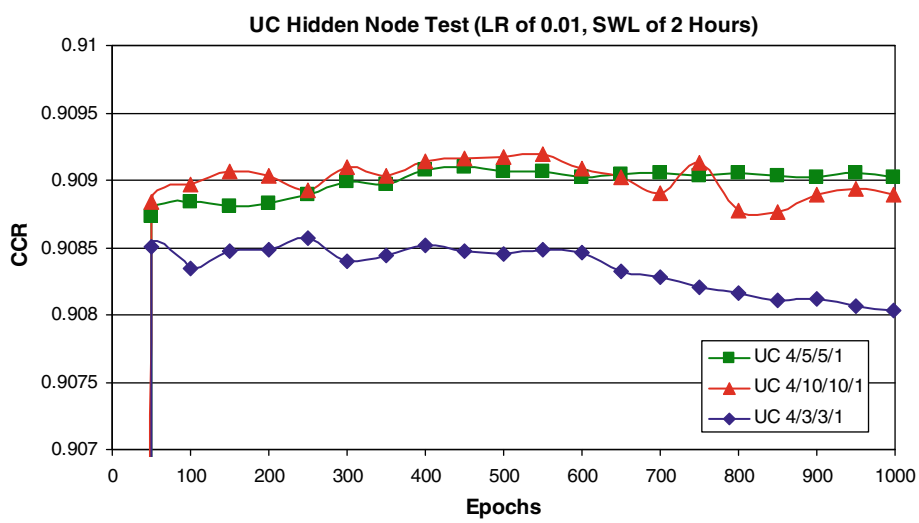

Fig. 6 Effect of hidden nodes for UC and 4 inputs

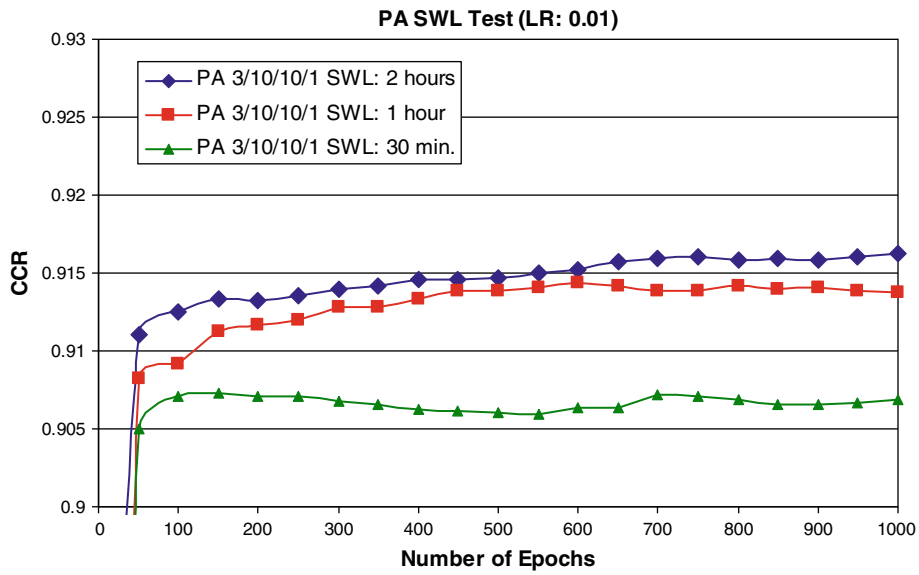

Fig. 7 Effect of *SWL* on PA 3/10/10/1

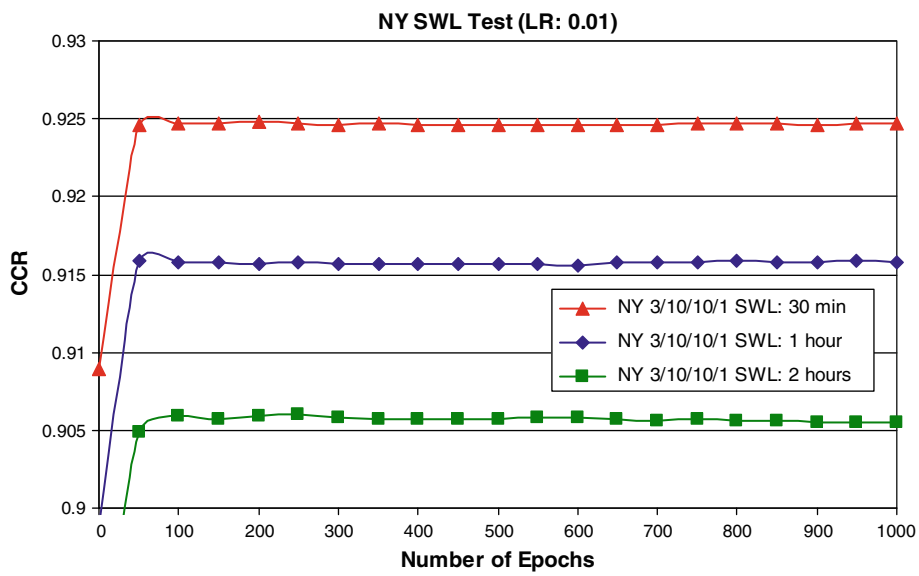

Fig. 8 Effect of *SWL* on NY 3/10/10/1

6 Performance metrics and emulation setup

Once our neural networks were trained and we verified our implementation in Squid, we ran several tests to measure the performance of how NNPCR-2 would fair against the other cache replacement strategies. We used two performance metrics, hit rate and byte hit rate. To emulate the request streams recorded in the trace files, we used our own version of *wget* [42]. This version is similar to *wget* except it did *not* write the data it received from the network to a file. This was done mainly because we had very limited space restrictions and also to decrease the training time by the amount of disk latency.

6.1 Performance metrics

Performance metrics are designed to measure the three different goals a web cache should meet: decrease user perceived lag, decrease overall bandwidth, and decrease load on original servers. Hit rate and byte hit rate are by far the most common, and stable measurements of web cache performance as a whole.

- *Hit rate*. This metric is used to measure all generic forms of caching. This is simply the number of *cache hits*, as defined in Sect. 2.1, that occur to the total number of *cacheable requests* seen by the proxy. It is

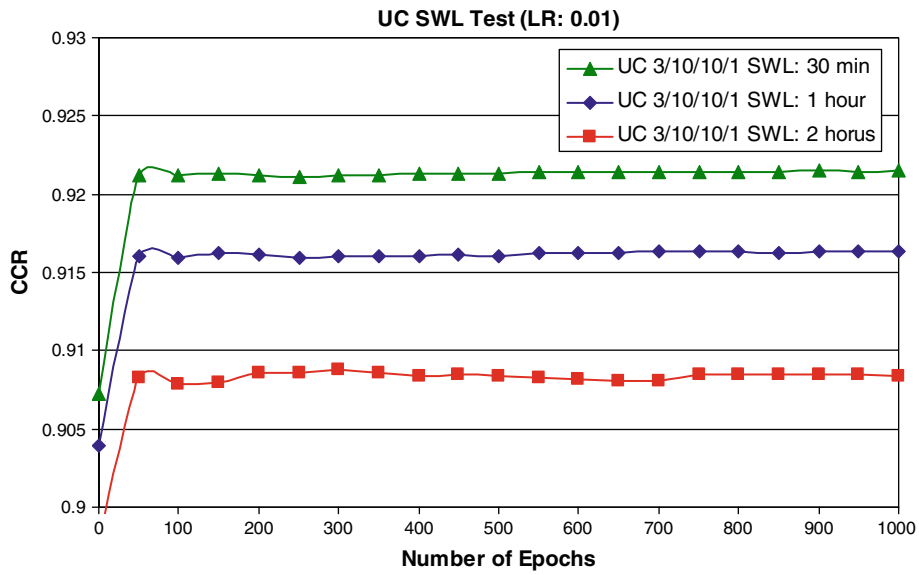

Fig. 9 Effect of *SWL* on UC 3/10/10/1

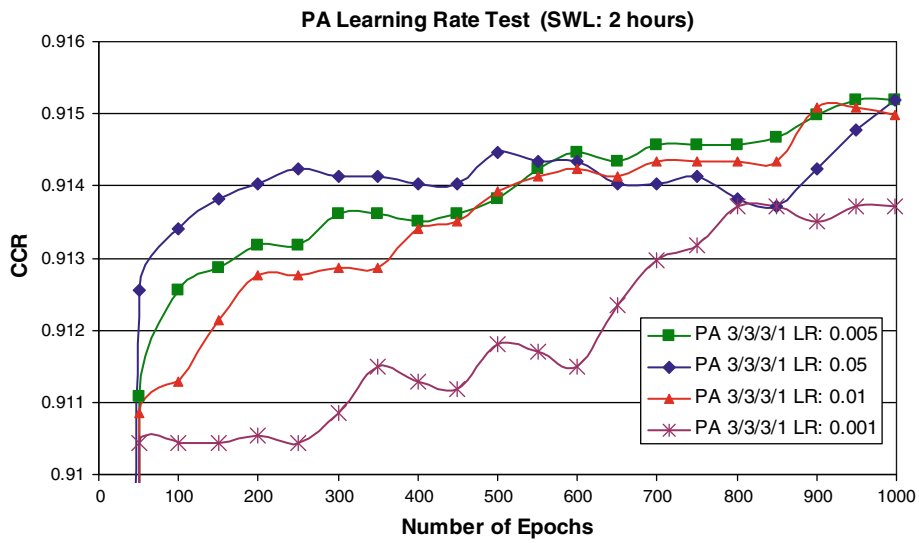

Fig. 10 Effect of learning rate on PA 3/3/3/1 Structure

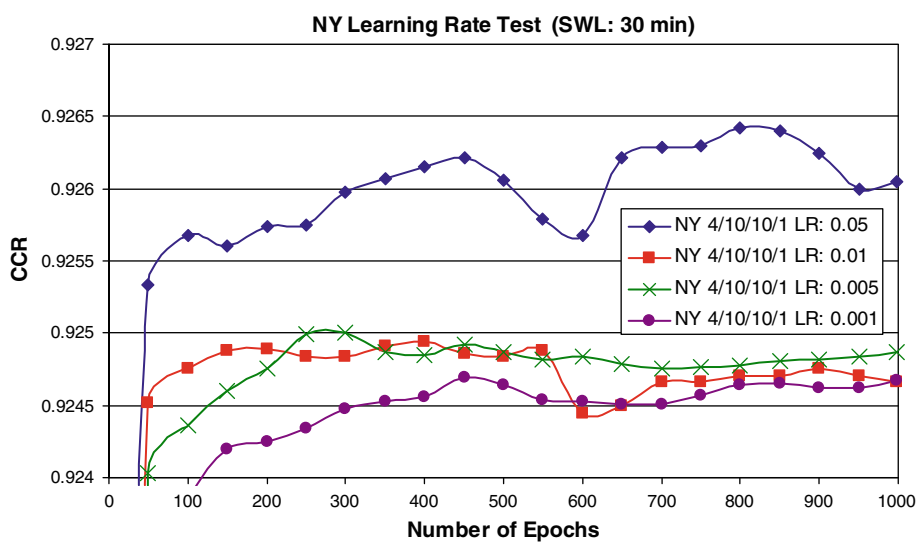

Fig. 11 Effect of learning rate on NY 4/10/10/1 Structure

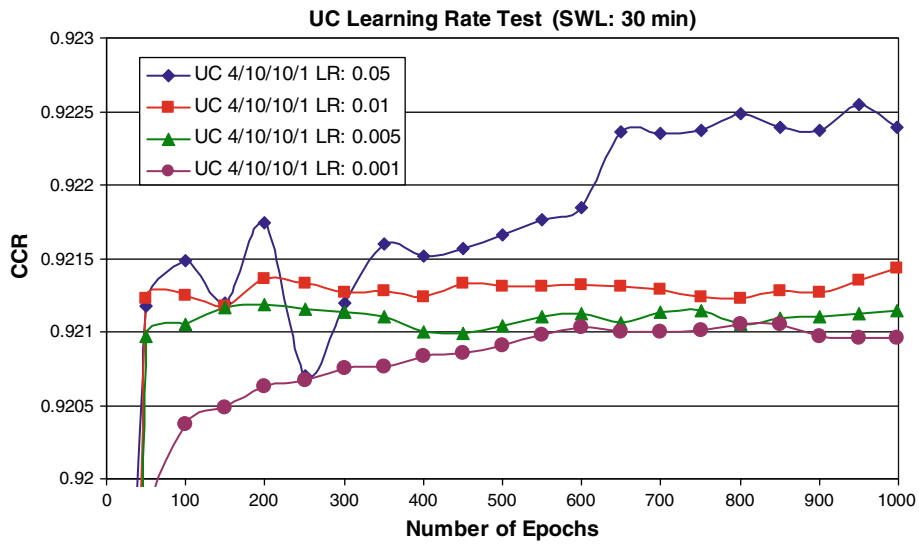

Fig. 12 Effect of learning rate on UC 4/10/10/1 Structure

important to realize that on average, 35–55% of the trace files we used were non-cacheable requests. Also important to note is that hit rate is relatively a good measure of the user perceived lag.

- *Byte hit Rate*. This metric is similar to hit rate, except it emphasizes the total bytes saved by caching certain objects. As a result, byte hit rate is an indicator of the total bandwidth we are saving for the proxy cache and origin servers. Letting h_i be the total number of bytes saved from all *cache hits* that occur for an object i and r_i be the total number of bytes for all *cacheable requests* to object i, and n, the total number of unique objects seen in a request stream, then the byte hit rate is

$$\frac{\sum_{i=1}^{n} h_i}{\sum_{i=1}^{n} r_i} \tag{8}$$

It is important to keep in mind that hit rate and the byte hit rate cannot be optimized for at the same time [1]. No cache replacement strategy can be deemed as the best because there is a tendency in request streams for smaller documents to be requested more often than larger ones due to the download time it takes to gather these objects. Strategies that optimize for hit rate typically give preference to objects in a smaller size range, but in doing so tend to decrease byte hit rate by giving less consideration to larger objects.

6.2 Trace files

In our experiment, we used trace files, flat-files with recorded web requests, to test each replacement strategy. The trace files were provided by IRCache [39]. IRCache gathers their trace files and other data on web caching from several different proxy servers located around the United States. More than enough information is provided in these files to indicate which requests were cacheable.

Originally, the trace files were provided with data spanning only a day of recorded web requests. We strung seven consecutive trace files together to create a week long trace file from each proxy that the data came from. Once this was done, we then 'cleaned' the files to have only the cacheable requests. We also exchanged each unique URI that appeared in the file with a unique integer identifier so that test initiation time could be decreased as well.

Table 4 presents statistics about the three traces we used for this emulation. Non-cacheable requests were extracted from the files prior to our experiment as mentioned earlier. Each trace represented varying levels of temporal locality, spatial locality, total bandwidth and number of requests testing the various limits of the replacement strategies. For both UC-Test and NY-Test, we used trace files spanning from April 3 to April 9, 2008 and for the PA-Test trace file, only an older set from July 19 to July 25, 2007 was available at the time of this research.

6.3 Emulation setup and parameters

Once our trace files were parsed, we first created all the necessary web objects that we would need for testing on a separate server from the web proxy server. We used the size of the request and the URI to designate unique objects. Then, for each object in each trace file, we created a unique file with the same number of bytes as recorded in the trace. This process was done by essentially just filling up the file with as many newlines (`'\n'`) as there were bytes due to the fact that each character is a single byte. Once finished, we created a *wget* link file, which essentially is a flat file with a set of URIs that is requested in order one at a time.

Table 4 Trace file statistics

Trace file location	Urbana-Champaign, Illinois (UC-Test)	New York, New York (NY-Test)	Palo Alto, California (PA-Test)
Total requests	2,223,555	1,767,052	519,232
Cacheable requests	884,928 (39.79%)	778,486 (44.06%)	141,466 (27.24%)
Total bytes	181.96 GB	21.65 GB	8.28 GB
Cacheable bytes	96.77%	82.72%	82.51%
Unique objects	1,050,160 (47.23%)	853,433 (48.29%)	269,476 (51.90%)
Cacheable objects	53.27% of above	54.02% of above	31.27% of above

There were two main issues with this approach, however. First, the files are requested one at a time, not asynchronously. Second, we could not control the timing of the requests so as to sync with the trace file durations, etc. This lead to our edited version of *wget* requesting the URIs as fast as it could on one thread, which leads to an entire week's worth of web requests to being finished in about 11 h of testing. Thus, certain characteristics of the request stream could not be replicated from the trace file into our emulation. Clearly, the recency values of requests/objects in our emulation did not sync up to what the recency values when the trace file was recorded. However, relative characteristics such as frequency and size would of course prevail in either testing environment.

In an attempt to correct the problem of correlating recency values as they would have been seen in the trace file (since the trace file is clearly an accurate model of the practical environment), we multiplied all durations, ΔT_i, by 25. This is equivalent to 1-week testing time/25 = 6.72 emulated testing hours; 6.72 h was close to the average total time that each link file completed within. In the implementation, this means anytime that we needed to measure the amount of time that had passed since the last request occurred or the recency of an object, we would scale it by 25 times what was actually being reported.

Prior to running each test, we warmed up the cache with a smaller trace file from Boulder, Colorado. By using another trace file different from the others, we could guarantee that no files from that trace run would conflict with the other trace files. As a result, the cache would be filled by the time we started our simulation, putting our cache replacement strategies in effect immediately upon starting our tests. Therefore, all the results presented in Sect. 7 are the full results of the cache replacement strategy.

7 Emulation results and observations

We performed several different tests comparing different instances of NNPCR-2. We also compared NNPCR-2 with the other cache replacement strategies. Most results reported here have a standard deviation of error within 0.2% for both metrics covered. Table 5 refers to the different parameters used for the various versions of NNPCR-2 tested. All NNPCR-2 tests used a max frequency of 128 and a size class of 22 (since we configured Squid to not cache objects larger than 8 MB as mentioned earlier). Overall, we found that the neural network trained with 4 inputs and UC testing file was the best instance of NNPCR-2 consistently on all trace files.

Table 5 Various instances of NNPCR-2 and their parameters

Test name	Structure	Training file	SWL
NNPCR4-UC	4/10/10/1	UC	30 min
NNPCR4-NY	4/10/10/1	NY	30 min
NNPCR3-UC	3/10/10/1	UC	30 min
NNPCR3-NY	3/10/10/1	NY	30 min
NNPCR3-PA	3/3/3/1	PA	2 h

Since we used the implementation of the heap class in Squid, M-Metric was implemented in a non-optimal way. The Squid implementation of the heap only updates the key upon a cache hit or when the object is added into the heap. It does not update again otherwise. As a result, many keys become deprecated rather quickly in M-Metric, and objects which had high values keep those same values until an object is either requested again or finally removed from the cache. In this implementation, M-Metric thus suffers greatly from cache pollution and its performance is severely hindered. However, M-Metric for this research really served more as a lower bound to test NNPCR-2 against and as a way for us to validate our own implementation. Thus, we *expected* M-Metric to have the worst performance metrics in all testing simulations. If it did not, then we knew that somehow our implementation was invalidated.

In this paper, we report the results of the Palo Alto tests. The New York and the Urbana-Champaign tests could be found in [32]. A summary of the results of PA, NY, and UC tests are provided in Tables 6 and 7.

The following figures present the running analysis of the cache hit rate in relation to the number of requests fired at a certain point in the test. As we can see initially in Figs. 13,

Table 6 Summary of hit rate results

	LRU (%)	GDSF (%)	LFU-DA (%)	M-Metric (%)
PA	+2.57	−4.32	−0.46	+48.8
NY	+1.98	−8.67	+7.67	+40.0
UC	+5.1	−9.75	+0.817	+54.4
Average	+3.22	−7.58	+2.68	+47.6

Table 7 Summary of byte hit rate results

	LRU (%)	GDSF (%)	LFU-DA (%)	M-Metric (%)
PA	+2.55	+16.25	−0.037	+45.0
NY	+5.03	+47.8	−1.98	+71.6
UC	+9.39	+54.7	−0.86	+72.4
Average	+5.66	+39.6	−1.07	+63.0

14, 15, 16, the graphs for different instances of NNPCR-2 and even other strategies have more or less the same shape for hit rate and byte hit rate. We can see where at certain points in the emulation a large number of requests were made that statistically offered more to the cache hit rate. Large jumps that occur in performance metrics at the same points are clearly demonstrating patterns in the request stream as opposed to the efficiency of the corresponding replacement strategy. However, despite the characteristics of the PA-Test file, there is still a difference between strategies as shown in Figs. 15 and 16.

In terms of the comparison of the NNPCR-2 instances, as shown in Figs. 13 and 14, the performance metrics seem to always be relatively the same throughout—so close that due to their standard deviation of error, it is too close to call which instances are clearly doing better by examining these graphs alone. However, in the results we saw by doing multiple test runs, NNPCR4-UC always outperformed the others, even if it was not by much.

The worst performance of NNPCR-2 occurs with NNPCR3-UC. Clearly, the lack of input of the size of the object was a large drop in performance all around. This pattern occurred across all tests we ran. Despite the drop in performance, NNPCR3-UC has the same curvature as the other instances and is also a little more stable. Notice between 35,000 and about 50,000 requests that the other strategies are still rising in hit rate, while NNPCR3-UC flattens out.

When comparing NNPCR4-UC with the other strategies, we saw that LFU-DA had almost the same performance in hit rate and byte hit rate (Figs. 15 and 16). LFU-DA outperforms NNPCR4-UC by approximately 0.2% in both metrics. GDSF had the best hit rate, but as a result suffered in byte hit rate. This is an expected effect due to GDSF's weighting toward smaller, frequently requested objects. Both LFU-DA and NNPCR4-UC had about a great balance between hit rate and byte hit rate, where LRU followed incredibly close behind them in both metrics. As expected, M-Metric had the worst performance.

The most interesting observation is how the strategies except M-Metric had the same performance within the first 20,000 requests. This demonstrates all 4 of these strategies ability to pick up extreme changes after performing the Boulder-Colorado warm up phase. However, as the strategies deviate, we can see the differences in these strategies. Again, this pattern occurs not only in the Palo Alto test runs, but also New York and Urbana-Champaign.

We summarize the results of our tests in the following two tables. Table 6 shows the percentages of the average hit rate of NNPCR-2 as it compares to the four other strategies, LRU, GDSF, LFU-DA, and M-Metric for the three trace files PA, NY, and UC. A "+" sign means that NNPCR-2 was on average better and a "−" sign means that it was worse. For example, For PA trace file, NNPCR-2, was on average 2.57% better than LRU and 4.32% worse than

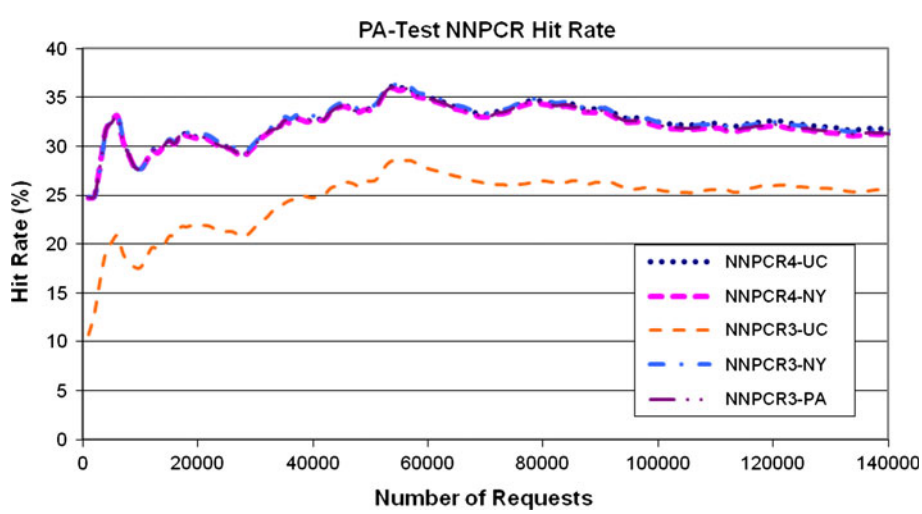

Fig. 13 PA-Test hit rate results for different instances of NNPCR-2

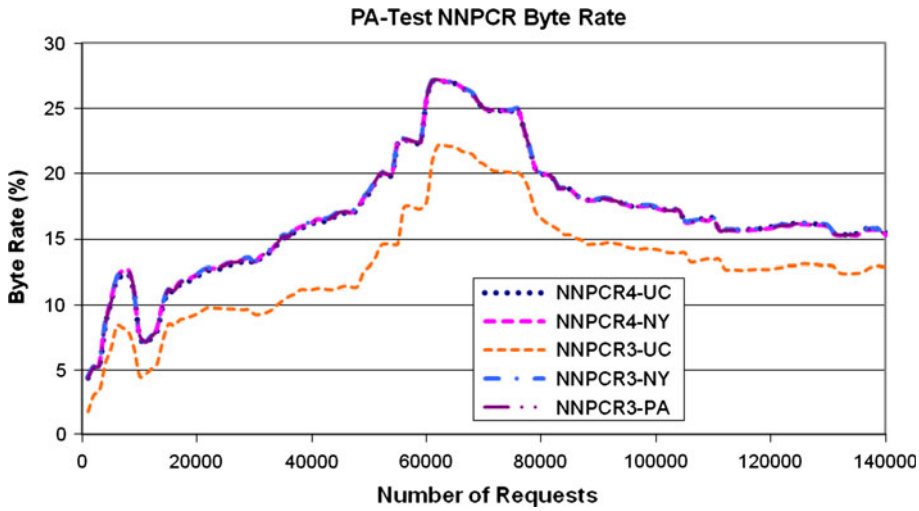

Fig. 14 PA-Test byte hit rate results for different instances of NNPCR-2

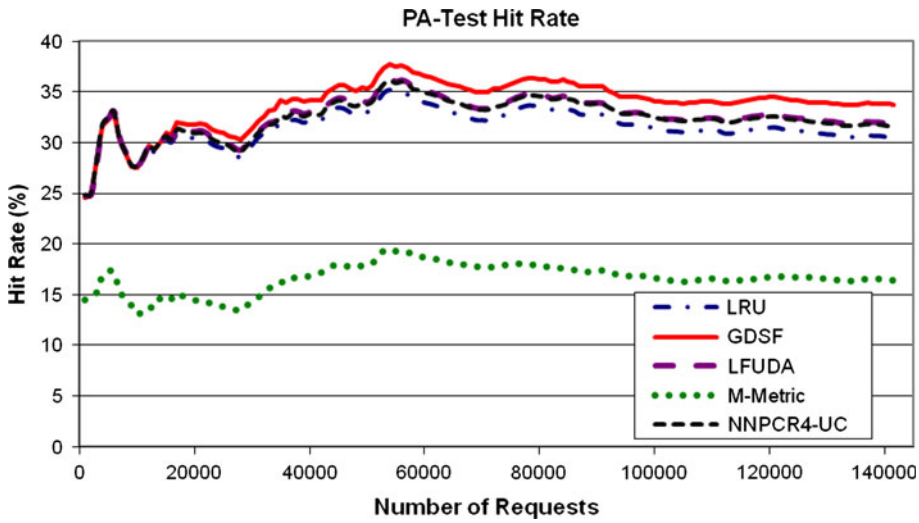

Fig. 15 PA-Test hit rate results for different replacement strategies

GDSF. The last shows the averages over all traces. Table 7 shows similar results for byte hit rate.

8 Conclusions

We implemented and tested NNPCR-2 in a real, practical web proxy cache framework. While NNPCR-2 may have not outperformed all the strategies, clearly NNPCR-2 was able to make better, balanced decisions than LRU and GDSF and had similar performance with LFU-DA. From our results, we see that NNPCR-2 is absolutely a plausible proxy cache replacement strategy. We have added an aging factor to NNPCR to deprecate objects from the cache in order to prevent cache pollution; this greatly improved NNPCR-2 and allowed it to run in an actual implementation for longer periods of time.

It was also shown that training NNPCR-2 is rather trivial, but multiple runs should always be done in order to find the correct sliding window length and other parameters to optimize NNPCR-2. We have also improved the training methods to gain an almost 5% increase in CCR from NNPCR. Besides the odd case of NNPCR3-UC, the neural network can actually generalize from small training sets, such as Palo Alto, to larger ones such as New York and Urbana-Champaign.

Although we trained on a day's worth of patterns and emulated a week's worth of requests, NNPCR-2 may handle even larger training and testing sets. Training the neural network is trivial and once trained, NNPCR-2 requires no additional parameter tuning and handles changes in the request stream characteristics quite well. It is absolutely possible to provide default neural network structures and already trained configurations of NNPCR-2 to end users and system administrators in ready-to-use configurations.

9 Future work

While we were able to reconfigure certain inputs (such as recency), there still needs to be more work done to see

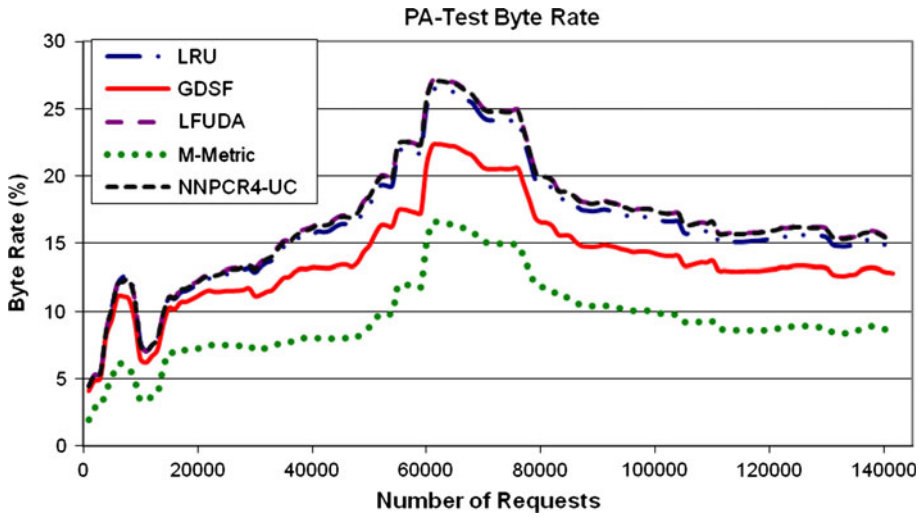

Fig. 16 PA-Test byte hit rate results for different replacement strategies

what information can be given to NNPCR-2 to allow it to make even better decisions. Clearly, it has a far greater potential than we originally realized. For instance, NNPCR-2 can abstract in such a way that clearly different training sets can be used to form the weights and yet still perform almost equally in all circumstances. Can we, for instance, provide request stream characteristics such as the rate of incoming requests into the neural network to use? What other aspects of web caching, or possibly caching in general allow the network to realize relationships that we may miss?

It may also be the case that the current training methods we are using are limiting how well the neural network can actually learn. We assume the cross-entropy objective function for training the neural network. However, cacheability of an object is more a probability in which we can map it to a range between [0.0, 1.0]. For instance, in regard to the SWL, what if we used the fractions of the sliding window length that two requests are from each other to measure how cacheable an object would be. In this case, the neural network would start to learn probabilities based on how often the request is made, and how much of the SWL it may take for the object to be tested again. With this method, the sum-of-squares error (SSE) function would need to be used instead, and rather than using CCR as a training metric, the error would have to be normalized as the root-mean-square error as a measurement. This area of expansion for NNPCR-2 is as open as the entire field of neural networks and artificial intelligence is allowed.

References

1. Podlipnig S, Boszormenyi L (2003) A survey of web cache replacement strategies. ACM Comput Surv 35(4):374–398
2. Davison B (2001) A web caching primer. IEEE Internet Comput 5(4):38–45
3. Romano S, ElAarag H (2008) A quantitative study of recency and frequency-based cache replacement strategies. Communication and Networking Simulation Symposium (CNS2008), Spring Simulation Multiconference, Ottawa, pp 70–78
4. Gan Q, Suel T (2009) Improved techniques for result caching in web search engines. In: Proceedings of the 18th international conference on World Wide Web, Spain, pp 431–440
5. Pallis G et al (2008) A Clustering-based prefetching scheme on a Web cache environment. Comput Electr Eng 34:309–323
6. Kumar C, Norris JB (2008) A new approach for a proxy-level web caching mechanism. Decis Support Syst 46(1):52–60
7. Chiang IR et al (2007) Periodic cache replacement policy for dynamic content at application server. Decis Support Syst 43:336–348
8. Ortiz JPG, Ruiz VG, Lopez MF, Garcia I (2008) Interactive transmission of JPEG2000 images using web proxy caching. IEEE Trans Multimedia 10(4):629–636
9. Liu J, Xu J (2004) Proxy caching for media streaming over the Internet. IEEE Commun Mag 42(8):88–94
10. Luo Q et al (2008) Form-based proxy caching for database-backed web sites: keywords and functions. VLDB J 17(3):489–513
11. Houtzager G, Jacob C, Williamson C (2006) An evolutionary approach to optimal web proxy cache placement. IEEE Congr Evolut Comput
12. Aguilar J, Leis EL (2006) A coherence-replacement protocol for web proxy cache systems. Int J Comput Appl 28(1):12–18
13. Fagni et al. (2006) Boosting the performance of web search engines: caching and prefetching query results by exploiting historical usage data. ACM Trans Inf Syst 24(1):51–78
14. Kaya CC et al (2009) An admission-control technique for delay reduction in proxy caching. Decis Support Syst 46(2):594–603
15. Katsaros D, Manolpoulos Y (2004) Web caching in broadcast mobile wireless environments. IEEE Internet Comput 8(3): 37–44
16. Kumar N et al (2007) Supporting mobile decision making with association rules and multi-layered caching. Decis Support Syst 43:16–30
17. Sabegi et al. (2006) Using fuzzy logic to improve cache replacement decisions. Int J Comput Sci Netw Secur, 6(3A)
18. Calzarossa MC, Valli G (2003) A fuzzy Algorithm for web caching. Simul Ser J 35(4):630–636

19. Venketesh P et al (2009) A survey on applications of neural networks and evolutionary techniques in web caching. IETE Tech Rev 26(3):171–180
20. Khalid H (1997) A new cache replacement scheme based on backpropagation neural networks. ACM SIGARCH Comput Archit News 25(1):27–33
21. Pomerene J, Puzak TR, Rechtschaffen R, Sporacio F (1984) Prefetching mechanism for a high-speed buffer store, US Patent
22. Khalid H (1997) Performance of the KORA-2 cache replacement scheme. ACM SIGARCH Comput Archit News 25(4):17–21
23. Obaidat MS, Khalid H (1998) Estimating neural networks-based algorithm for adaptive cache replacement. IEEE Trans Syst Man Cybern B Cybern 28(4):602–611
24. Khalid H, Obaidat MS (1999) Application of neural networks to cache replacement. Neural Comput Appl J 8:246–256
25. Arlitt MF, Cherkasova L, Dilley J, Friedrich RJ, Jin TY (2000) Evaluating content management techniques for web proxy caches. ACM SIGMETRICS Perform Eval Rev 27(4):3–11
26. Bahn H, Koh K, Min SL, Noh SH (2002) Efficient replacement of nonuniform objects in web caches. IEEE Comput 35:65–73
27. Cao P, Irani S (1997) Cost-aware WWW proxy caching algorithms. In: Proceedings of USENIX symposium on internet technologies and systems, pp 193–206
28. Rhea SC, Liang K, Brewer E (2003) Value-based web caching. In: Proceedings of the 12th international conference on World Wide Web. ACM Press, pp 619–628)
29. Caceres R, Douglis F, Feldmann A, Glass G, Rabinovich M (1998) Web proxy caching: the devil is in the details. ACM SIGMETRICS Perform Eval Rev 26(3):11–15
30. Rizzo L, Vicisano L (2000) Replacement policies for a proxy cache. IEEE/ACM Trans Netw 8(2):158–170
31. Cobb W, ElAarag H (2008) Web proxy cache replacement scheme based on back propagation neural network. J Syst Softw 81:1539–1558 doi:10.1016/j.jss.2007.10.024
32. ElAarag H, Romano S (2009) Improvement of the neural network proxy cache replacement strategy. In: Communication and networking simulation symposium (CNS2009), Spring simulation multiconference, San Diego, pp 22–27
33. Wessels D (1995) Intelligent caching for World-Wide-Web objects. M.S. thesis, University of Colorado at Boulder, Boulder, CO
34. "Hypertext Transfer Protocol—HTTP/1.1", Available at http://www.w3.org/Protocols/rfc2616/rfc2616.html
35. "What is Squid?", Available at http://www.squid-cache.org/Intro/
36. Dilley J, Arlitt M, Perret S (1999) Enhancement and validation of Squid's cache replacement policy. Internet Systems and Applications Laboratory, HP Laboratories, Palo Alto, California, HPL-1999-69
37. Tatarinov I (1998) An efficient LFU-like policy for Web caches. Tech. Rep. NDSU-CSORTR-98-01, Computer Science Department, North Dakota State University, Wahpeton, ND
38. Aggarwal CC, Wolf JL, Yu PS (1999) Caching on the world wide web. IEEE Trans Knowl Data Eng 11(1):94–107
39. "IRCache Home", Available at http://www.ircache.net/
40. Reed RD, Marks RJ (1999) Neural smithing, supervised learning in feedforward artificial neural networks. The MIT Press, Massachusetts, pp 1–53
41. ElAarag H, Romano S (2009) Training of NNPCR-2: an improved neural network proxy cache replacement strategy. In: 2009 International symposium on performance evaluation of computer and telecommunication systems (SPECTS 2009), vol 41, pp 260–267
42. "GNU Wget", Available at http://www.gnu.org/software/wget/